Programación y robótica

Nombre: _____ Fecha: _____

Qué vas a aprender

- Diseñar algoritmos con diagramas de flujo y pseudocódigo y traducirlos a un lenguaje de programación
- Usar variables, tipos de datos, operadores, estructuras de control, funciones y listas
- Programar un microcontrolador para leer sensores y gobernar actuadores
- Analizar robots móviles, sistemas IoT y el impacto ético y de seguridad de los sistemas programados

Para pensar

Un robot aspirador limpia toda la casa
sin mapa previo y vuelve solo a su base.
¿Hay alguien dentro que piense?

Explicación

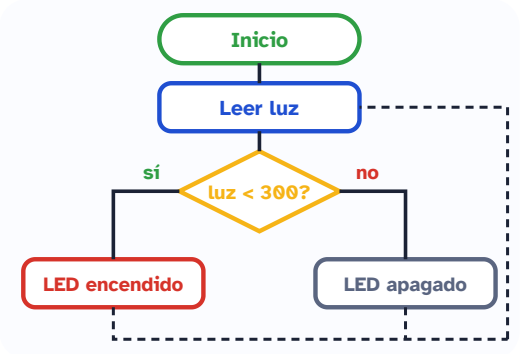
1.1 Algoritmos, diagramas de flujo y pseudocódigo

Un **algoritmo** es una secuencia finita, ordenada y precisa de instrucciones que resuelve un problema. Debe terminar, cada paso debe estar definido sin ambigüedad y debe producir un resultado. Una receta de cocina o las instrucciones de montaje de un mueble son algoritmos para personas; un programa es un algoritmo escrito para una máquina.

Antes de programar conviene diseñar el algoritmo con herramientas independientes del lenguaje:

- **Diagramas de flujo**: usan símbolos normalizados unidos por flechas. Óvalos para el inicio y el fin, rectángulos para los procesos, rombos para las decisiones (con dos salidas, sí y no) y romboides para la entrada y salida de datos.
- **Pseudocódigo**: una descripción en lenguaje casi natural con estructura de programa: «si luz < 300 entonces encender LED; si no, apagar LED».

Diseñar bien un algoritmo implica **descomponer** el problema en partes más sencillas, reconocer **patrones** que se repiten, quedarse con lo esencial (**abstracción**) y ordenar los pasos. Este modo de razonar, el **pensamiento computacional**, sirve también fuera de la informática. Un mismo problema puede resolverse con varios algoritmos; se elige el más claro, el más rápido o el que menos memoria consume.



1.2 Lenguajes y entornos de programación

Los procesadores solo entienden **código máquina**: secuencias de ceros y unos específicas de cada procesador. Escribir así es lentísimo, por eso se usan

lenguajes de programación:

- **De bajo nivel** (ensamblador): cada instrucción corresponde casi a una del procesador. Muy rápidos, pero difíciles y poco portables.
- **De alto nivel** (Python, C, C++, Java, JavaScript): cercanos al lenguaje humano y válidos para distintos procesadores.
- **Visuales o por bloques** (Scratch, ArduinoBlocks, MakeCode): se encajan piezas gráficas. Evitan errores de sintaxis y son ideales para empezar.

Para que el procesador ejecute un programa de alto nivel hay que traducirlo. Un **compilador** traduce todo el programa de una vez a código máquina y genera un ejecutable (C, C++, el lenguaje de Arduino). Un **intérprete** traduce y ejecuta instrucción a instrucción mientras el programa funciona (Python, JavaScript): más cómodo para probar, algo más lento.

Los programas se escriben en un **entorno de desarrollo** (IDE), que incluye editor con colores, compilador, depurador y herramientas para cargar el programa en la placa. Los equipos profesionales usan además sistemas de **control de versiones** como Git, que guardan el historial de cambios y permiten que varias personas trabajen en el mismo código.



1.3 Variables, tipos de datos y operadores

Una **variable** es un espacio de memoria con un **nombre** en el que se guarda un dato que puede cambiar durante el programa. Una **constante** guarda un valor fijo, como el número de un pin. Cada variable tiene un **tipo de dato**, que indica qué valores puede guardar y cuánta memoria ocupa:

- **Enteros** (int): números sin decimales, como 25 o –3.
- **Reales** o de coma flotante (float): números con decimales, como 21,5.
- **Booleanos** (bool): verdadero o falso.

- **Caracteres** y **cadenas de texto** (char, String): «A», «Hola».

En lenguajes como C++ hay que declarar el tipo; en Python se deduce al asignar el valor. La **asignación** se escribe con un solo signo igual: `temp = 21.5`.

Los **operadores** permiten hacer cálculos y comparaciones:

- **Aritméticos**: + – * / y % (resto de la división entera: 17 % 5 = 2).
- **De comparación**: == (igual), != (distinto), <, >, <=, >=. Dan un resultado booleano.
- **Lógicos**: and (&&), or (||) y not (!), que combinan condiciones igual que el álgebra de Boole.

Hay que vigilar la división entre enteros: en C++, 7 / 2 da 3, porque se descartan los decimales. Los nombres de variables deben ser claros: `distanciaCm` mejor que `x`.

Tipos de datos

Tipo	Ejemplo	Uso
int	25	contadores
float	21.5	medidas
bool	true	estados
String	"Hola"	mensajes

1.4 Estructuras de control

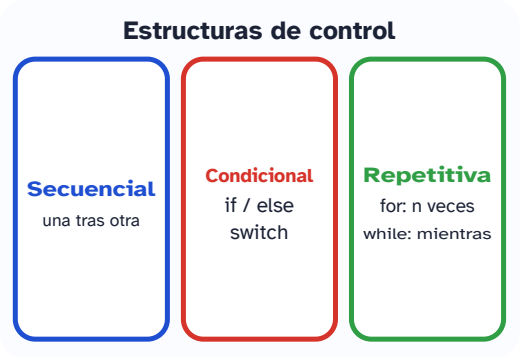
Todo programa se construye con tres estructuras básicas, demostró en 1966 el teorema de la programación estructurada:

- **Secuencial**: las instrucciones se ejecutan una detrás de otra.
- **Condicional** o selectiva: se ejecuta un bloque u otro según una condición. **if** (si), **else** (si no) y **else if** (si no, si...), o **switch** para elegir entre muchos casos.
- **Repetitiva** o bucle: se repite un bloque. El bucle **for** repite un número conocido de veces; el **while** (mientras) repite mientras se cumpla una condición, que se comprueba al principio.

```
for (int i = 0; i < 5; i++) {
  parpadear();          // se repite 5 veces
```

```
}
while (distancia > 20) {
  avanzar();            // mientras no haya obstáculo
  distancia = medir();
}
```

Un error típico es el **bucle infinito**: un while cuya condición nunca deja de cumplirse porque dentro no cambia nada. En los microcontroladores, en cambio, el programa principal es un bucle infinito a propósito: la función `loop()` de Arduino se repite mientras la placa tenga alimentación. Las estructuras se pueden **anidar**: un if dentro de un for, un bucle dentro de otro. La **sangría** del código muestra qué instrucciones están dentro de cada bloque.



2.1 Funciones, listas y modularidad

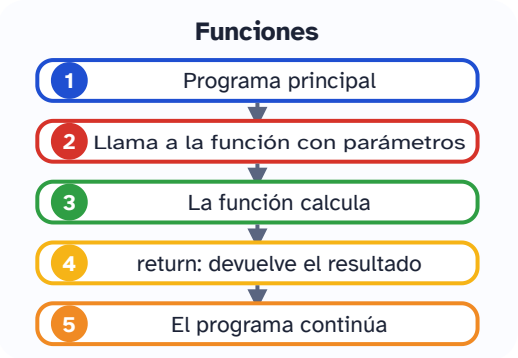
Una **función** es un bloque de código con nombre que realiza una tarea concreta. Se define una vez y se **llama** tantas veces como haga falta. Puede recibir **parámetros** (datos de entrada) y **devolver** un resultado con `return`:

```
float celsiusAFahrenheit(float c) {
  return c * 9.0 / 5.0 + 32;
}
float f = celsiusAFahrenheit(25);  // f = 77
```

Dividir un programa en funciones es la **programación modular**: el código es más corto, más legible, más fácil de probar y de reutilizar. Cada función debe hacer una sola cosa y tener un nombre que lo diga. Las variables declaradas dentro de una función son **locales** (solo existen dentro de ella); las declaradas fuera son **globales**.

Para manejar muchos datos del mismo tipo se usan **vectores** o **listas** (arrays): una variable con varias posiciones numeradas desde `0`. Por ejemplo, `notas[0]`

es el primer elemento. Con un bucle for se recorren todos para calcular una media, buscar un máximo u ordenarlos. Las librerías son colecciones de funciones ya escritas por otros (para manejar una pantalla, un servo o un sensor) que se incluyen en el programa y ahorran mucho trabajo.



2.2 Depuración, pruebas y buenas prácticas

Casi ningún programa funciona a la primera. Los errores (*bugs*) son de tres tipos:

- **De sintaxis**: el código no respeta las reglas del lenguaje (falta un punto y coma o una llave). El compilador los detecta y señala la línea.
- **De ejecución**: el programa compila pero falla al ejecutarse, por ejemplo al dividir entre cero o acceder a una posición inexistente de una lista.
- **Lógicos**: el programa funciona pero hace algo distinto de lo previsto (un < en lugar de un <=). Son los más difíciles de encontrar.

La **depuración** consiste en localizarlos y corregirlos. Técnicas útiles: imprimir el valor de las variables en el monitor serie, ejecutar paso a paso con un depurador, probar cada función por separado y explicar el código en voz alta a otra persona (o a un patito de goma).

Las **pruebas** comprueban el programa con casos normales, casos límite (el valor justo en el umbral, una lista vacía) y casos erróneos (un sensor desconectado). Un buen programa además está **comentado** (explica el porqué, no lo obvio), usa nombres claros, evita números mágicos con constantes con nombre, tiene una sangría coherente y se guarda con control de versiones. El código se lee muchas más veces de las que se escribe.

Errores

Error	Ejemplo	Lo detecta
Sintaxis	falta ;	compilador
Ejecución	dividir entre 0	al ejecutar
Lógico	< por <=	las pruebas

2.3 Microcontroladores y placas de desarrollo

Un **microcontrolador** es un pequeño ordenador completo en un chip: procesador, memoria de programa (flash), memoria de datos (RAM), temporizadores, conversor analógico-digital y pines de entrada y salida. Está en lavadoras, mandos, coches, juguetes y relojes. Las **placas de desarrollo** como Arduino, micro:bit, Raspberry Pi Pico o ESP32 facilitan su uso: incluyen regulador de tensión, conexión USB y pines accesibles.

Un programa de Arduino tiene dos funciones obligatorias: `setup()`, que se ejecuta una vez al encender (configurar pines, iniciar la comunicación), y `loop()`, que se repite indefinidamente:

```
const int LED = 9;            // pin del LED
void setup() {
  pinMode(LED, OUTPUT);
  Serial.begin(9600);         // monitor serie
}
void loop() {
  int luz = analogRead(A0);   // 0 a 1023
  Serial.println(luz);
  if (luz < 300) digitalWrite(LED, HIGH);
  else digitalWrite(LED, LOW);
  delay(100);
}
```

Una **Raspberry Pi** no es un microcontrolador sino un ordenador con sistema operativo, útil para tareas más pesadas (visión artificial, servidores). El **ESP32** añade wifi y Bluetooth, por lo que es muy usado en proyectos de Internet de las cosas.


Microcontrolador
Arduino, ESP32
sin sistema operativo
tiempo real
bajo consumo
Miniordenador
Raspberry Pi
con Linux
más potencia
más consumo

2.4 Entradas, salidas y comunicación

Los pines de un microcontrolador pueden configurarse de varias formas:

- **Entrada digital**: `digitalRead(pin)` lee HIGH o LOW (pulsadores, finales de carrera, sensores todo-nada). Necesitan resistencias de **pull-up** o **pull-down** para no quedar flotando; muchas placas incluyen pull-up internas (`INPUT_PULLUP`). Los pulsadores mecánicos rebotan y dan varios pulsos: se filtra por programa (*debounce*).
- **Salida digital**: `digitalWrite(pin, HIGH)` pone el pin a 5 V o 3,3 V. Da solo unos 20 mA: suficiente para un LED, no para un motor.
- **Entrada analógica**: `analogRead(pin)` usa el ADC para leer una tensión (en Arduino Uno, de 0 a 1023 para 0-5 V). Sirve para potenciómetros, LDR, termistores y sensores con salida de tensión. La función `map()` convierte la lectura a la escala deseada.
- **Salida PWM**: `analogWrite(pin, valor)` con valores de 0 a 255 regula el brillo de un LED o la velocidad de un motor a través de un driver.

Para comunicarse con otros dispositivos se usan protocolos: **serie** (UART), con el ordenador o con módulos Bluetooth; **I2C**, con dos cables para muchos sensores y pantallas; y **SPI**, más rápido, para tarjetas SD. En lugar de `delay()`, que detiene todo el programa, en proyectos complejos se usa `millis()`, que permite hacer varias tareas a la vez comprobando el tiempo transcurrido.

Entradas y salidas

Función	Tipo	Rango
digitalRead	entrada	0 / 1
digitalWrite	salida	0 / 1
analogRead	entrada ADC	0-1023
analogWrite	salida PWM	0-255

3.1 Robots móviles: estructura y movimiento

Un **robot** es una máquina programable que percibe su entorno con sensores, toma decisiones y actúa sobre él con cierta autonomía. Sus subsistemas son la **estructura** o chasis, los **actuadores** (motores y su driver), los **sensores**, el

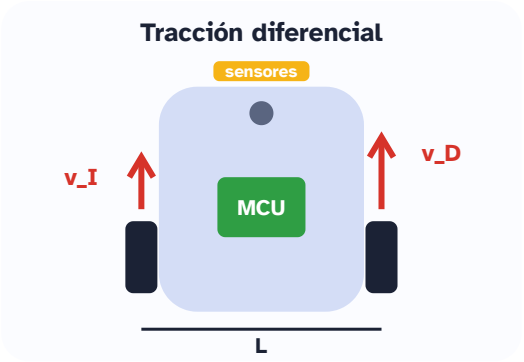
controlador (microcontrolador y programa), la **alimentación** (baterías) y, a veces, la **comunicación**.

Los robots móviles más sencillos usan **tracción diferencial**: dos ruedas motrices independientes y una rueda loca. Si las dos giran igual, el robot avanza recto; si una gira más rápido, gira hacia el lado de la lenta; si giran en sentidos contrarios, gira sobre sí mismo. Su velocidad lineal y angular son:

$$v = (v_D + v_I) / 2 \cdot \omega = (v_D - v_I) / L$$

siendo v_D y v_I las velocidades de las ruedas derecha e izquierda y L la distancia entre ellas. La velocidad de cada rueda depende de su diámetro y de su giro: $v = \pi \cdot d \cdot n / 60$ (n en rpm).

Los sensores típicos son de **infrarrojos** para detectar líneas en el suelo, de **ultrasonidos** para medir distancias a obstáculos, **bumpers** de choque, **encoders** en las ruedas para medir el recorrido (odometría) y una **unidad inercial** (acelerómetro y giróscopo) para conocer la orientación. Los robots avanzados combinan cámaras y LiDAR para construir un mapa y localizarse en él a la vez.



3.2 Algoritmos para robots: seguidores, obstáculos y estados

Un **seguidor de líneas** usa dos o más sensores infrarrojos que distinguen el negro de la línea del blanco del suelo. El algoritmo más sencillo es todo-nada: si el sensor izquierdo ve la línea, girar a la izquierda; si la ve el derecho, girar a la derecha; si ninguno, seguir recto. Con más sensores se calcula la **posición** de la línea y un control **proporcional** (o PID) ajusta la diferencia de velocidad entre ruedas: el robot va más rápido y oscila menos.

Un robot que **evita obstáculos** mide la distancia con ultrasonidos: si es menor que un umbral, se detiene, gira y vuelve a medir.

Cuando el comportamiento es complejo se programa como una máquina de estados: el robot está siempre en un estado (avanzar, girar, buscar, cargar) y pasa a otro cuando ocurre un evento. Se implementa con una variable que guarda el estado actual y un `switch` en el bucle principal:

```
switch (estado) {
  case AVANZAR:
    avanzar();
    if (distancia() < 20) estado = GIRAR;
    break;
  case GIRAR:
    girar();
    if (distancia() > 40) estado = AVANZAR;
    break;
}
```

Es la versión programada del GRAFCET y hace el código claro y fácil de ampliar.



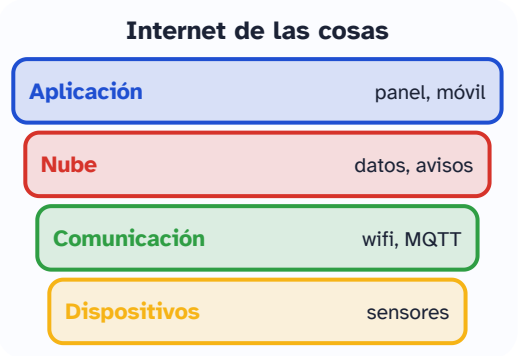
3.3 Internet de las cosas y la nube

El **Internet de las cosas** (IoT) conecta a internet objetos cotidianos e industriales para que envíen datos y reciban órdenes: termostatos, contadores de agua, sensores de un campo de cultivo, máquinas de una fábrica, farolas o pulseras de actividad. Un sistema IoT tiene varias capas:

- **Dispositivos**: sensores y actuadores con un microcontrolador que tiene conexión (wifi, Bluetooth, LoRa, redes móviles).
- **Comunicación**: los datos viajan con protocolos ligeros como **MQTT**, en el que los dispositivos publican mensajes en «temas» y otros se suscriben a

ellos a través de un servidor llamado *broker*, o mediante peticiones **HTTP** a un servidor web.
- **Nube**: servidores que almacenan los datos, los procesan y generan avisos.
- **Aplicación**: paneles web o móviles para visualizar los datos y mandar órdenes.

El IoT permite el mantenimiento predictivo, la agricultura de precisión, las ciudades inteligentes o la domótica. Sus retos son el **consumo energético** de millones de dispositivos (se diseñan para dormir la mayor parte del tiempo y funcionar años con una pila), la **interoperabilidad** entre fabricantes y, sobre todo, la **seguridad** y la **privacidad** de los datos que recogen.



3.4 Inteligencia artificial, ciberseguridad y ética

La **inteligencia artificial** (IA) permite que los sistemas aprendan a partir de datos en lugar de seguir reglas escritas una a una. En el **aprendizaje automático**, un modelo se entrena con muchos ejemplos (miles de fotos de piezas buenas y defectuosas) y después clasifica casos nuevos. Hoy se usa en visión artificial para inspeccionar productos, en robots que aprenden a agarrar objetos, en asistentes de voz y en la conducción asistida. Incluso pequeños microcontroladores pueden ejecutar modelos sencillos.

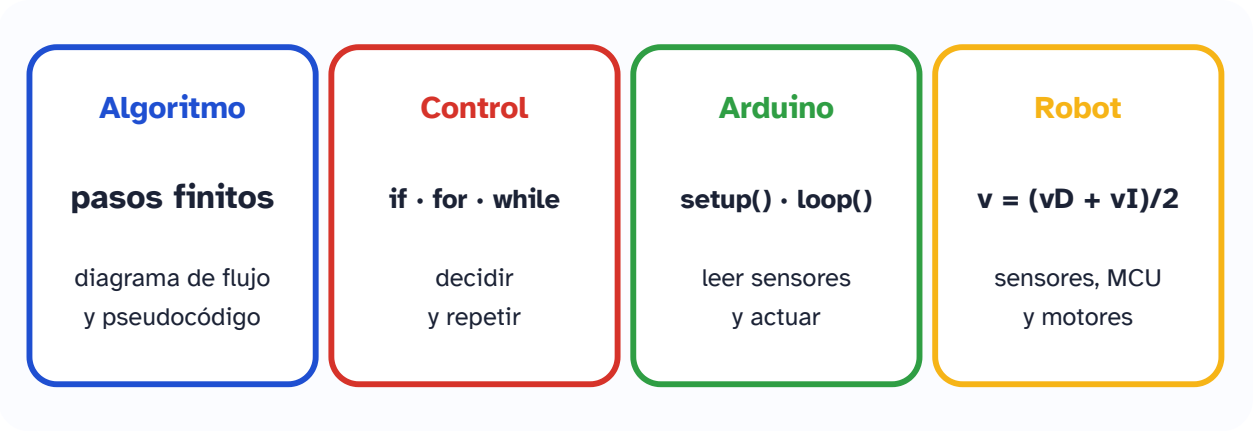
Todo sistema conectado puede ser atacado. La **ciberseguridad** exige cambiar las contraseñas por defecto, actualizar el software, cifrar las comunicaciones, dar a cada dispositivo solo los permisos necesarios y separar las redes de los dispositivos IoT. Una cámara o un enchufe inteligente mal protegidos pueden usarse para espiar o para atacar otros sistemas.

Programar sistemas que actúan en el mundo real tiene implicaciones **éticas**: ¿quién es responsable si un robot o un coche autónomo causa un daño? ¿Están

los datos de entrenamiento libres de **sesgos** que discriminen a ciertas personas? ¿Se respeta la **privacidad**, como exige el Reglamento General de Protección de Datos? ¿Es accesible para todos? La Unión Europea regula la IA según su nivel de riesgo. Un buen ingeniero piensa en estas preguntas desde el principio del diseño.



Ideas clave de la programación y la robótica



Ejercicios

1. **Define algoritmo y nombra sus tres características.**

2. **Dibuja (describe) con símbolos el diagrama de flujo para decidir si un número es par.**

3. **Diferencia compilador e intérprete con un ejemplo de lenguaje para cada uno.**

4. **Indica el tipo de dato de: número de alumnos, altura en metros, ¿está encendido?, nombre.**

5. **Calcula: 23 % 4, 15 / 4 (enteros) y 15.0 / 4.**

6. **Evalúa con x = 4: (x > 2) && (x < 10), (x == 5) || (x != 3), !(x > 3).**

7. **¿Cuántas veces se repite for (int i = 1; i <= 6; i++)?**

8. **Escribe un bucle que muestre los números del 10 al 1.**

9. **Clasifica el error: falta una llave de cierre; el programa divide entre una variable que vale 0; el robot gira a la derecha cuando debía girar a la izquierda.**

10. **¿Qué hacen setup() y loop()?**

11. **¿Qué rango de valores da analogRead y qué rango acepta analogWrite en Arduino Uno?**

12. **Nombra cuatro subsistemas de un robot móvil.**

13. **Escribe en pseudocódigo un algoritmo que pida 10 números y muestre el mayor.**

14. **Escribe una función que reciba una distancia en cm y devuelva true si es menor que 20.**

15. **¿Qué valor tiene s al terminar? s = 0; for (i = 1; i <= 5; i++) s = s + i * i;**

16. **Encuentra el error lógico: if (temp > 30) encender(); if (temp < 30) apagar(); ¿qué pasa si temp = 30?**

17. **Programa la lectura de un pulsador en el pin 7 con pull-up interna para encender un LED en el pin 13 mientras está pulsado.**

18. **Una LDR da lecturas entre 100 (oscuro) y 900 (luz). Escribe la instrucción que convierte la lectura a un porcentaje de luz.**

19. **Calcula el valor de analogWrite para que un motor reciba el 75 % de su tensión.**

20. **Escribe el algoritmo de un seguidor de líneas con dos sensores (I y D, 1 = ve la línea).**

21. **Ruedas de 7 cm a 90 rpm. ¿Cuánto tarda el robot en recorrer 2 m en línea recta?**

22. **¿Por qué es mejor millis() que delay() en un robot que debe vigilar un sensor mientras hace parpadear un LED?**

23. **Describe la máquina de estados de una puerta automática con sensor de presencia.**

24. **Explica cómo funciona MQTT con el ejemplo de un sensor de temperatura y un móvil.**

25. **Indica tres medidas de seguridad para una cámara IP doméstica.**

26. **Escribe una función que ordene de menor a mayor una lista de n números (método de la burbuja).**

27. **Programa un control proporcional para un seguidor de líneas con un sensor que da la posición de la línea entre –100 (izquierda) y 100 (derecha). Velocidad base 150 y Kp = 0,8.**

28. **Un robot debe girar 90° sobre sí mismo con ruedas a ±20 cm/s y L = 15 cm. ¿Cuánto tiempo deben girar los motores?**

29. **Con encoders de 20 pulsos por vuelta en ruedas de 6,5 cm, ¿cuántos pulsos corresponden a 1 m de recorrido? ¿Qué resolución tiene la medida?**

30. **Implementa con millis() un LED que parpadee cada 500 ms sin bloquear el programa.**

31. **Un nodo IoT consume 80 mA durante 2 s cada vez que envía un dato y 0,05 mA dormido. Envía un dato cada 15 minutos. ¿Cuánto dura una batería de 2000 mA·h?**

32. **Diseña la máquina de estados de un robot aspirador con los estados LIMPIAR, ESQUIVAR, VOLVER_BASE y CARGAR, indicando los eventos.**

33. **Explica qué es el sesgo en un sistema de IA y propone un ejemplo y una solución en un sistema de control de acceso con reconocimiento**

facial.

34. **Calcula cuántos bytes ocupa guardar cada minuto, durante un día, la temperatura (float, 4 bytes) y la hora (unsigned long, 4 bytes) en un registrador de datos.**

35. **Debate: ¿quién debería ser responsable si un robot de reparto autónomo atropella a un peatón? Argumenta considerando a fabricante, programador, empresa y usuario.**

Practica

1. **Un algoritmo debe ser...**

   A) infinito y ambiguo
   B) finito, ordenado y preciso
   C) escrito en Python
   D) siempre gráfico

2. **En un diagrama de flujo, el rombo representa...**

   A) el inicio
   B) una decisión
   C) un proceso
   D) la salida de datos

3. **¿Qué traduce todo el programa de una vez a código máquina?**

   A) Un intérprete
   B) Un compilador
   C) Un depurador
   D) Un editor

4. **El tipo de dato adecuado para una temperatura de 21,5 °C es...**

   A) int
   B) float
   C) bool
   D) char

5. **17 % 5 vale...**

   A) 3,4
   B) 2
   C) 3
   D) 12

6. **En C++, 7 / 2 con enteros da...**

   A) 3,5
   B) 3
   C) 4
   D) error

7. **¿Qué operador compara si dos valores son iguales?**

A) =

B) ==

C) !=

D) <=

8. **Para repetir un bloque un número conocido de veces se usa...**

   A) if

   B) for

   C) switch

   D) return

9. **Un bucle while cuya condición nunca cambia provoca...**

   A) un error de sintaxis

   B) un bucle infinito

   C) una función

   D) una lista

10. **La instrucción return sirve para...**

    A) repetir

    B) devolver un resultado desde una función

    C) leer un pin

    D) comentar

11. **El primer elemento de una lista notas es...**

    A) notas[1]

    B) notas[0]

    C) notas[-1]

    D) notas.primero

12. **Un < escrito en lugar de un <= es un error...**

    A) de sintaxis

    B) de ejecución

    C) lógico

    D) de compilación

13. **En Arduino, la función que se ejecuta una sola vez al encender es...**

    A) loop()

    B) setup()

    C) main()

D) start()

14. **analogRead en un Arduino Uno devuelve valores entre...**

    A) 0 y 1

    B) 0 y 255

    C) 0 y 1023

    D) 0 y 5

15. **analogWrite(pin, 128) genera...**

    A) una tensión exacta de 2,5 V

    B) una señal PWM al 50 %

    C) una lectura

    D) un error

16. **¿Por qué un motor no se conecta directamente a un pin de Arduino?**

    A) Por el color

    B) Porque el pin da solo unos 20 mA

    C) Porque es digital

    D) Porque gira al revés

17. **En un robot de tracción diferencial, si las ruedas giran en sentidos contrarios a la misma velocidad...**

    A) avanza

    B) gira sobre sí mismo

    C) retrocede

    D) se para

18. **Un sensor de ultrasonidos mide...**

    A) el color

    B) la distancia por el eco

    C) la temperatura

    D) la inclinación

19. **En MQTT, los dispositivos publican mensajes en...**

    A) temas a través de un broker

    B) correos electrónicos

    C) páginas web

    D) tarjetas SD

20. **¿Cuál es una buena práctica de seguridad IoT?**

A) Mantener la contraseña de fábrica

B) Cambiar contraseñas y actualizar el software

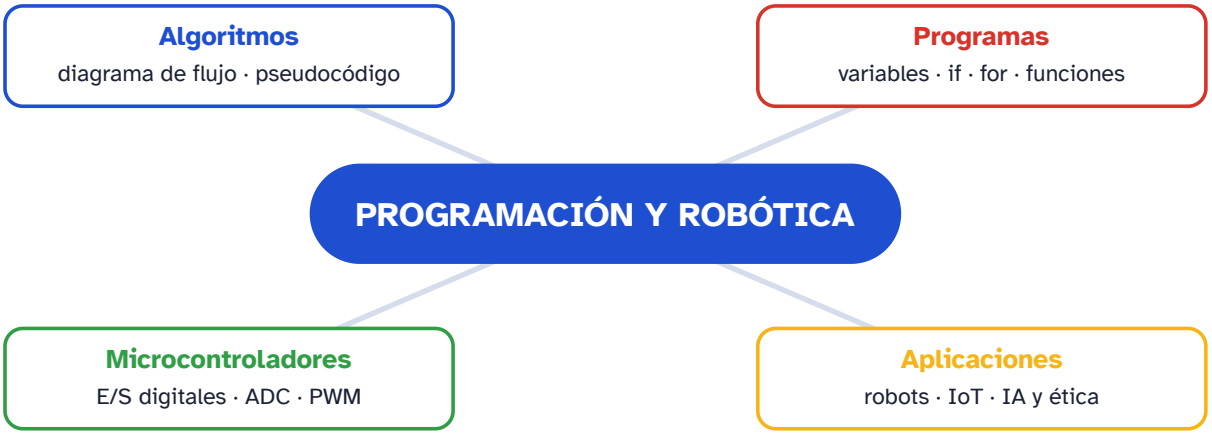
C) Abrir todos los puertos

D) No cifrar

Explícalo con tus palabras

Incluye: Qué es un algoritmo y cómo se dibuja un diagrama de flujo; Qué diferencia hay entre un bucle for y un while; Para qué sirven las funciones y los parámetros; Qué hacen setup() y loop() en Arduino; Cómo gira un robot de tracción diferencial; Qué riesgos de seguridad tiene un dispositivo IoT y cómo evitarlos.

Resumen



Glosario

Algoritmo	Secuencia finita y ordenada de pasos que resuelve un problema.
Diagrama de flujo	Representación gráfica de un algoritmo con símbolos normalizados.
Pseudocódigo	Descripción de un algoritmo en lenguaje casi natural.
Compilador	Programa que traduce todo el código fuente a código máquina.
Intérprete	Programa que traduce y ejecuta el código instrucción a instrucción.
Variable	Espacio de memoria con nombre cuyo valor puede cambiar.
Tipo de dato	Clase de valores que puede guardar una variable.
Condicional	Estructura que ejecuta un bloque u otro según una condición.
Bucle	Estructura que repite un bloque de instrucciones.
Función	Bloque de código con nombre que realiza una tarea y puede devolver un valor.
Lista o vector	Variable con varias posiciones numeradas desde 0.
Depuración	Proceso de localizar y corregir errores en un programa.
Microcontrolador	Ordenador completo en un chip para controlar dispositivos.

PWM	Señal de pulsos cuyo ciclo de trabajo regula una potencia media.
Tracción diferencial	Sistema de dos ruedas motrices independientes que gira variando sus velocidades.
Máquina de estados	Modelo en el que el sistema está en un estado y cambia con eventos.
Internet de las cosas	Red de objetos conectados que envían datos y reciben órdenes.
MQTT	Protocolo ligero de mensajes por temas para IoT.

Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

Algoritmo

Secuencia finita, ordenada y precisa de pasos que resuelve un problema.

Símbolos del diagrama de flujo

Óvalo: inicio/fin. Rectángulo: proceso. Rombo: decisión.

Compilador e intérprete

Compilador: traduce todo de una vez. Intérprete: instrucción a instrucción.

Tipos de datos

int, float, bool, char y String.

Operador %

Resto de la división entera: 17 % 5 = 2.

Estructuras de control

Secuencial, condicional (if, switch) y repetitiva (for, while).

Función

Bloque con nombre, parámetros y return; se reutiliza.

Tipos de errores

Sintaxis, ejecución y lógicos.

Arduino

setup() una vez; loop() siempre.

Entradas y salidas

digitalRead/Write (0-1), analogRead (0-1023), analogWrite PWM (0-255).

Tracción diferencial

$v = (v_D + v_I)/2$; $\omega = (v_D - v_I)/L$.

IoT

Dispositivos conectados que envían datos a la nube (wifi, MQTT).

Soluciones

1. **B)** Debe terminar y cada paso debe estar claro.
2. **B)** Tiene dos salidas: sí y no.
3. **B)** El intérprete traduce instrucción a instrucción.
4. **B)** Tiene decimales.
5. **B)** El operador % da el resto de la división entera.
6. **B)** La división entera descarta los decimales.
7. **B)** Un solo = es asignación.
8. **B)** while se usa cuando no se sabe cuántas veces.
9. **B)** El programa no sale nunca del bucle.
10. **B)** Termina la función y entrega el valor.
11. **B)** Los índices empiezan en 0.
12. **C)** El programa funciona pero hace algo distinto.
13. **B)** loop() se repite indefinidamente.
14. **C)** ADC de 10 bits.
15. **B)** 128 de 255 es aproximadamente el 50 %.
16. **B)** Hace falta un driver o un transistor.
17. **B)** v = 0 y ω máxima.
18. **B)** Mide el tiempo de ida y vuelta del sonido.
19. **A)** Otros dispositivos se suscriben a esos temas.
20. **B)** Las contraseñas por defecto son públicas.

Soluciones de los ejercicios

1. Secuencia de instrucciones para resolver un problema; debe ser finito, ordenado y preciso (sin ambigüedad).
2. Inicio (óvalo) → leer N (romboide) → ¿N % 2 == 0? (rombo) → sí: escribir «par»; no: escribir «impar» (romboides) → fin (óvalo).
3. El compilador traduce todo el programa de una vez y genera un ejecutable (C++, Arduino). El intérprete traduce y ejecuta línea a línea (Python).
4. int, float, bool y String.
5. 3, 3 y 3,75.
6. true, true y false.
7. 6 veces (i = 1 a 6).
8. for (int i = 10; i >= 1; i--) { Serial.println(i); }
9. Sintaxis, ejecución y lógico.
10. setup() se ejecuta una vez al iniciar (configuración); loop() se repite continuamente (programa principal).
11. analogRead: 0 a 1023. analogWrite: 0 a 255.
12. Estructura, actuadores (motores), sensores, controlador y alimentación (y comunicación).
13.
```
mayor = leer()
repetir 9 veces:
  n = leer()
  si n > mayor: mayor = n
escribir(mayor)
```
14.
```
bool hayObstaculo(float d) {
  return d < 20;
}
```
15. s = 1 + 4 + 9 + 16 + 25 = 55.
16. Con temp = 30 no se cumple ninguna condición y el ventilador se queda como estaba. Hay que usar else o >= en una de ellas (o una histéresis).
17.
```
void setup() {
  pinMode(7, INPUT_PULLUP);
  pinMode(13, OUTPUT);
}
void loop() {
  if (digitalRead(7) == LOW) digitalWrite(13, HIGH);
  else digitalWrite(13, LOW);
}
```
18. int luz = map(lectura, 100, 900, 0, 100); (y conviene limitarla con constrain(luz, 0, 100)).
19. 0,75 · 255 ≈ 191.

20.
```
si I == 1 y D == 0: girar a la izquierda
si no, si I == 0 y D == 1: girar a la derecha
si no, si I == 1 y D == 1: avanzar recto (cruce)
si no: buscar la línea (girar despacio)
```

21. v = π · 7 · 90 / 60 = 33 cm/s. t = 200 / 33 = 6,1 s.
22. delay() detiene todo el programa: durante la espera no se lee el sensor. Con millis() se comprueba si ha pasado el tiempo sin bloquear, y el bucle sigue leyendo sensores.
23. Estados: CERRADA, ABRIENDO, ABIERTA, CERRANDO. CERRADA → ABRIENDO si detecta presencia; ABRIENDO → ABIERTA al final de carrera; ABIERTA → CERRANDO tras 5 s sin presencia; CERRANDO → ABRIENDO si detecta presencia, o → CERRADA al final de carrera.
24. El sensor publica la temperatura en el tema «casa/salon/temp» en un broker. La app del móvil está suscrita a ese tema y recibe cada nuevo valor. Para mandar una orden, el móvil publica en «casa/salon/calefaccion» y el actuador, suscrito, la ejecuta.
25. Cambiar la contraseña por defecto por una robusta, actualizar el firmware y conectarla a una red separada (o desactivar el acceso remoto si no se usa), además de usar comunicaciones cifradas.

26.
```
void ordenar(int v[], int n) {
  for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - 1 - i; j++) {
      if (v[j] > v[j + 1]) {
        int aux = v[j];
        v[j] = v[j + 1];
        v[j + 1] = aux;
      }
    }
  }
}
```

27.
```
int error = posicion();          // −100 a 100
int correccion = 0.8 * error;
int vIzq = 150 + correccion;
int vDer = 150 - correccion;
analogWrite(MOTOR_IZQ, constrain(vIzq, 0, 255));
analogWrite(MOTOR_DER, constrain(vDer, 0, 255));
```

28. ω = (20 − (−20)) / 15 = 2,67 rad/s. 90° = π/2 = 1,571 rad. t = 1,571 / 2,67 = 0,59 s.
29. Perímetro = π · 6,5 = 20,4 cm. Por pulso: 1,02 cm. 1 m = 100 / 1,02 ≈ 98 pulsos. Resolución ≈ 1 cm.

30.
```
unsigned long antes = 0;
bool estado = false;
void loop() {
  if (millis() - antes >= 500) {
    antes = millis();
    estado = !estado;
```

```
    digitalWrite(13, estado);
  }
  // aquí se pueden leer sensores sin esperas
}
```

31. Consumo por ciclo de 900 s: 80 · 2 + 0,05 · 898 = 160 + 44,9 = 204,9 mA·s. Media = 204,9 / 900 = 0,228 mA. Duración = 2000 / 0,228 ≈ 8780 h ≈ 1 año.
32. LIMPIAR → ESQUIVAR si choca u obstáculo cerca; ESQUIVAR → LIMPIAR tras girar un ángulo aleatorio; LIMPIAR → VOLVER_BASE si batería < 15 %; VOLVER_BASE → CARGAR al detectar la base; CARGAR → LIMPIAR con batería al 100 %.
33. Es un error sistemático aprendido de datos de entrenamiento no representativos. Si el sistema se entrena casi solo con fotos de un grupo de personas, fallará más con otras, que no podrán entrar. Solución: datos variados y equilibrados, pruebas por grupos y una alternativa de acceso manual.
34. 1440 registros · 8 bytes = 11 520 bytes ≈ 11,3 kB: cabe en la memoria de una tarjeta SD, pero no en los 2 kB de RAM de un Arduino Uno.
35. La respuesta debe valorar que el fabricante responde de defectos de diseño y de seguridad; la empresa operadora, del mantenimiento y del uso correcto; y que el marco legal europeo tiende a exigir seguros obligatorios, trazabilidad (registro de decisiones) y supervisión humana en sistemas de alto riesgo.